



NATIONAL UNIVERSITY

of Computer & Emerging Sciences

- **Students Name:** Mussa Raza, Muhammad Zain, Nisar Ahmad
- **Roll Number:** 24I-3022, 24I-3126, 24I-3131
- **Department:** Software Engineering
- **Section:** A
- **Assignment:** 1
- **Course:** Artificial Intelligence
- **Date:** 27/09/2026

1. Introduction and Problem Setup

In artificial intelligence, search algorithms are fundamental techniques used by problem-solving agents to compute an optimal sequence of actions from an initial configuration to a desired goal state. The UC Berkeley Pacman framework provides an interactive, discrete grid environment where Pacman must navigate mazes of varying topological complexity, avoid static wall obstacles, and locate food pellets or corner objectives. This project implements, evaluates, and compares five classical search algorithms: Depth-First Search (DFS), Breadth-First Search (BFS), Uniform-Cost Search (UCS), Greedy Best-First Search (GBFS), and A* Search. Additionally, complex multi-goal search spaces and admissible heuristics are developed for the Corners Problem and Food Search Problem.

1.1 Formal State Space Formulation

A classical search problem is formally formulated as a 5-tuple: $(S, s_0, \text{ACTIONS}(s), \text{RESULT}(s, a), \text{GOAL-TEST}(s), c(s, a, s'))$. Within the Pacman environment, these components correspond to the following:

1. State Space (S): For single-goal pathfinding, a state is defined as an integer coordinate pair (x, y) representing Pacman's exact position on the grid. For multi-goal problems, the state space is expanded to include historical progress, such as a tuple containing $(\text{position}, \text{visited_corners})$ or $(\text{position}, \text{food_grid})$.
2. Start State (s_0): The initial coordinates of Pacman when the layout is loaded from file, such as (x_start, y_start) .
3. Actions / Successor Function: From any valid state s , Pacman can attempt movement in four cardinal directions: North $(0, 1)$, South $(0, -1)$, East $(1, 0)$, and West $(-1, 0)$. The successor function $\text{getSuccessors}(s)$ returns a list of tuples: $(\text{next_state}, \text{action}, \text{step_cost})$, excluding any action that would cause Pacman to collide with a wall grid cell.
4. Goal Test ($\text{isGoalState}(s)$): A boolean predicate that determines whether state s satisfies the termination criteria. In single-goal mazes, it verifies whether $s == \text{goal_position}$. In multi-goal mazes, it verifies whether all required targets (such as all four corners or all food dots) have been collected.
5. Path Cost Function: The accumulated sum of step costs along an action sequence. In standard position search problems, every step has a uniform unit cost of $c(s, a, s') = 1$. In directional cost problems (such as `StayEastSearch`), step costs vary as a function of spatial coordinates, such as $\text{cost} = 0.5^x$.

1.2 Architecture and Codebase Separation

The project strictly maintains clean architectural boundaries between student-implemented search logic and untouched framework support files. The student code is contained entirely within two files:

- `search.py`: Contains generic graph search algorithms that operate on any abstract `SearchProblem`.

These include `depthFirstSearch`, `breadthFirstSearch`, `uniformCostSearch`, `greedyBestFirstSearch` (aliased as `gbfs`), `aStarSearch`, and the automated CSV trace logging utility.

- `searchAgents.py`: Defines concrete search problem formulations and heuristic evaluation functions.

These include `CornersProblem`, `cornersHeuristic`, `AnyFoodSearchProblem`, and `foodHeuristic`.

All other files in the distribution (`pacman.py`, `game.py`, `util.py`, `graphicsDisplay.py`) serve as the

underlying simulation engine and were left completely unmodified. All frontier data structures (Stack, Queue, PriorityQueue) are imported strictly from util.py without reliance on external Python collections.

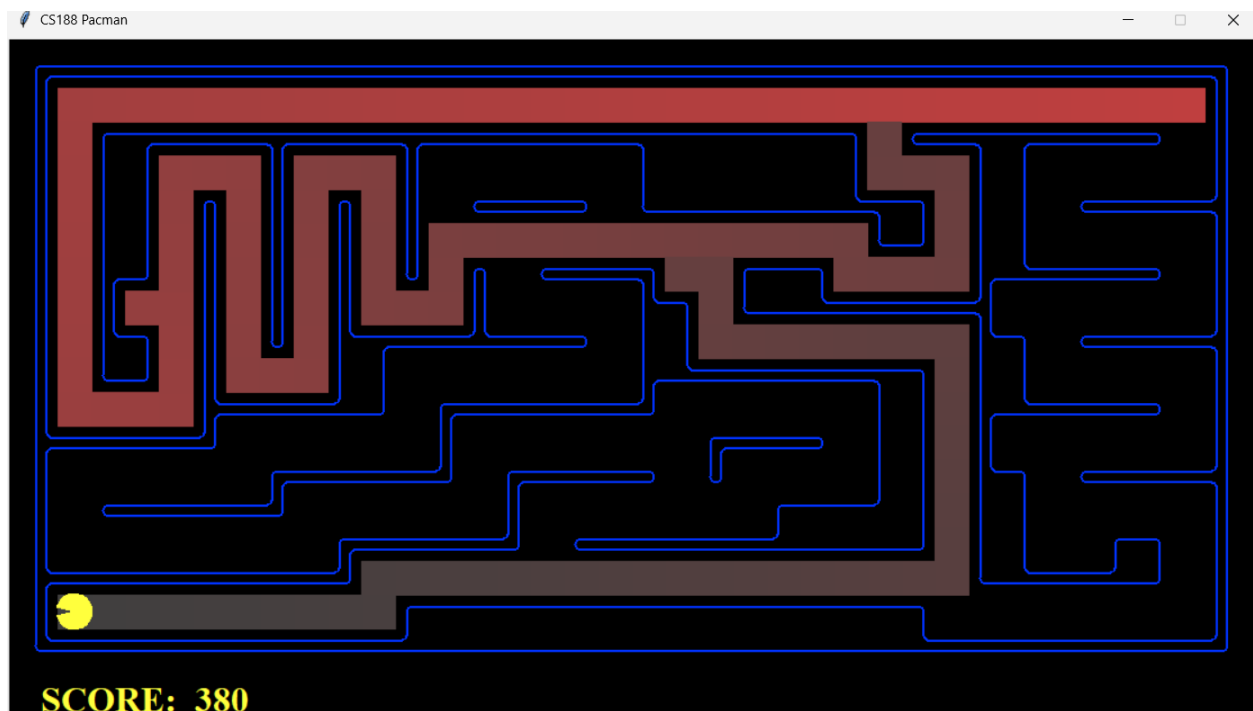
2. Implementation of Core Search Algorithms

Graph search algorithms maintain two primary data structures: the frontier (fringe), which stores candidate states generated but not yet expanded, and the explored set (visited), which tracks states that have already been expanded. To prevent infinite loops in cyclic grid graphs, our implementations follow the strict principle that goal checks occur when a node is removed from the frontier, not when it is generated, ensuring correct path reconstruction.

2.1 Depth-First Search (DFS) - Task 1

Depth-First Search explores the deepest unexpanded node first using a Last-In, First-Out (LIFO) stack (util.Stack). In accordance with assignment guidelines, successor nodes are pushed to the stack in reverse order of generation so that the mandatory expansion order follows North, East, South, West.

- Time Complexity: $O(b^m)$, where b is the branching factor (up to 4 in a grid) and m is the maximum depth of the state space.
- Space Complexity: $O(b * m)$, as DFS only needs to store the current branch and unexpanded siblings.
- Completeness: Complete in finite state spaces with cycle checking, but can be incomplete in infinite state spaces.
- Optimality: Non-optimal. DFS returns the first path found that reaches a goal, often taking long, winding detours.



```
PS E:\S5\AI\Project\pacman-search> python pacman.py -l mediumMaze -p SearchAgent -a fn=dfs
[SearchAgent] using function dfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 130 in 0.1 seconds
Search nodes expanded: 146
Pacman emerges victorious! Score: 380
Average Score: 380.0
Scores:      380.0
Win Rate:    1/1 (1.00)
Record:      Win
```

Figure 1: Solution path and terminal execution output for Depth-First Search (DFS) on mediumMaze.

2.2 Breadth-First Search (BFS) - Task 2

Breadth-First Search explores the shallowest unexpanded node first using a First-In, First-Out (FIFO) queue (util.Queue). BFS expands nodes level by level across the search tree.

- Time Complexity: $O(b^d)$, where d is the depth of the shallowest goal.
- Space Complexity: $O(b^d)$, because all nodes at depth d must be retained in memory.
- Completeness: Complete in all state spaces with a finite branching factor.
- Optimality: Strictly optimal for problems where all step costs are identical and positive ($c = 1$). Because BFS explores states in non-decreasing order of depth, the first time a goal state is popped, its path depth is guaranteed to be minimal.



Figure 2: Solution path and terminal execution output for Breadth-First Search (BFS) on mediumMaze.

2.3 Uniform-Cost Search (UCS) - Task 3

Uniform-Cost Search generalizes BFS to graphs with non-uniform edge costs by expanding the node with the lowest cumulative backward path cost $g(n)$ first. It utilizes a priority queue (`util.PriorityQueue`).

A critical requirement in UCS is handling priority updates. If a newly generated successor points to a state that is already in the frontier, but the new path offers a strictly lower cost $g(n')$, the priority queue entry must be updated. Our implementation tracks the best known cost to each state in a dictionary (`best_g`). When a cheaper path is discovered, `priorityQueue.update(state, new_cost)` is called, which replaces the existing item priority.

- Time and Space Complexity: $O(b^{(1 + \lfloor C^* / \epsilon \rfloor)})$, where C^* is the cost of the optimal solution and ϵ is the minimum step cost.

- Completeness: Complete as long as every action cost is strictly greater than zero ($c \geq \epsilon > 0$).

- Optimality: Strictly optimal. Nodes are expanded in monotonically increasing order of $g(n)$. When the goal state is popped, no remaining path in the frontier can achieve a lower cost.



Figure 3: Solution path and terminal execution output for Uniform-Cost Search (UCS) on mediumMaze.

2.4 Greedy Best-First Search (GBFS) - Task 4

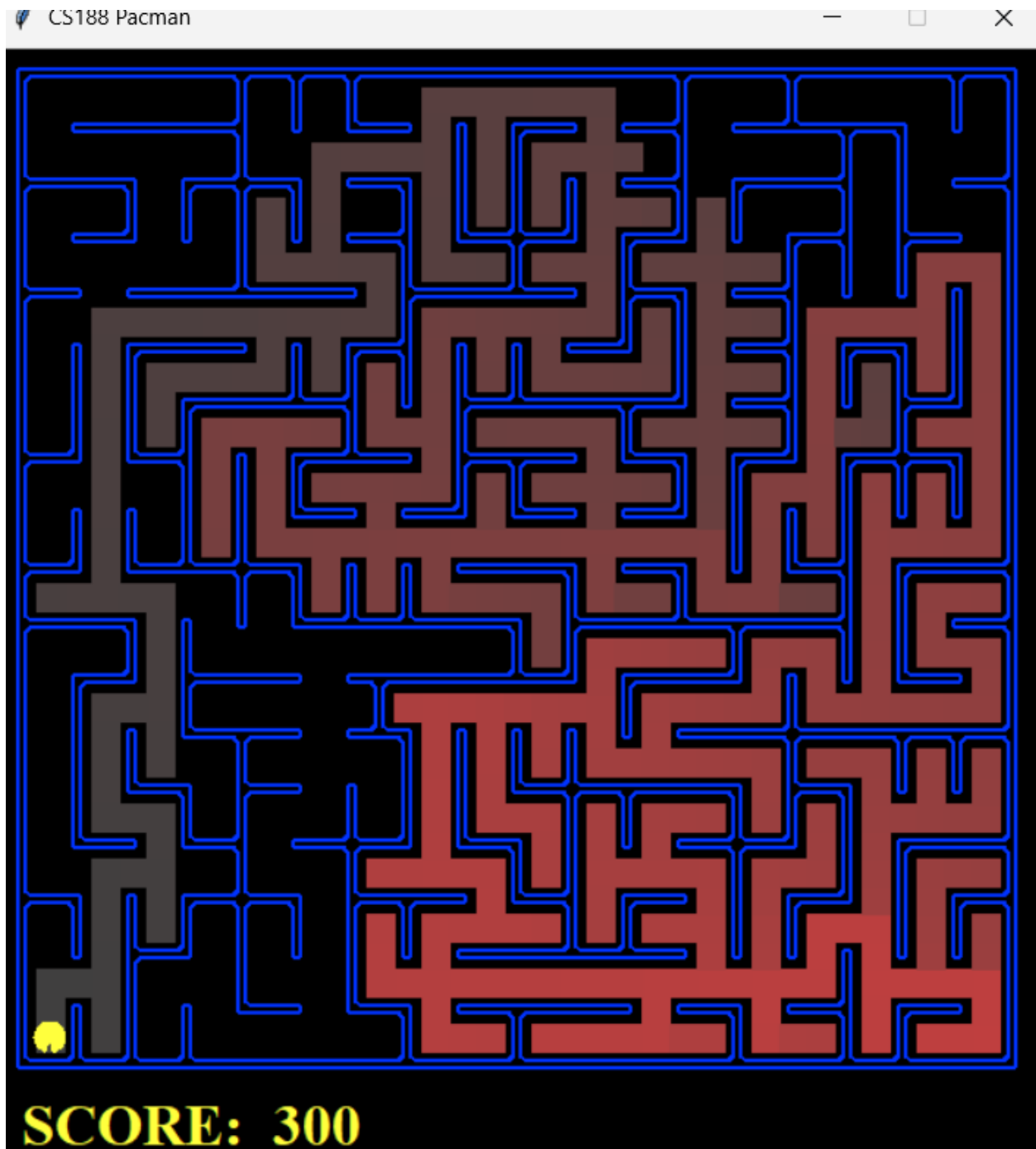
Greedy Best-First Search evaluates nodes strictly by an estimate of remaining distance to the goal, using the evaluation function $f(n) = h(n)$. It utilizes `util.PriorityQueue` ordered by $h(n)$, ignoring the accumulated cost $g(n)$.

Our implementation supports dynamic command-line heuristic passing, allowing users to specify heuristics such as `manhattanHeuristic` or `euclideanHeuristic` via the argument string: `-a fn=gbfs,heuristic=manhattanHeuristic`.

- Time and Space Complexity: In the worst case, $O(b^m)$, where m is the maximum depth. With a perfect

heuristic, complexity drops to $O(b * d)$.

- Completeness: Incomplete in infinite state spaces without cycle checking, but complete in finite graphs with an explored set.
- Optimality: Non-optimal. Because GBFS ignores $g(n)$, it is easily misled by local heuristic gradients, often plunging into deep dead-end pockets that appear physically close to the goal.



```
PS E:\S5\AI\Project\pacman-search> python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=gbfs,heuristic=manhattanHeuristic
[SearchAgent] using function gbfs and heuristic manhattanHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 210 in 0.2 seconds
Search nodes expanded: 466
Pacman emerges victorious! Score: 300
Average Score: 300.0
Scores: 300.0
Win Rate: 1/1 (1.00)
Record: Win
```

Figure 4: Solution path and terminal execution output for Greedy Best-First Search (GBFS) on bigMaze.

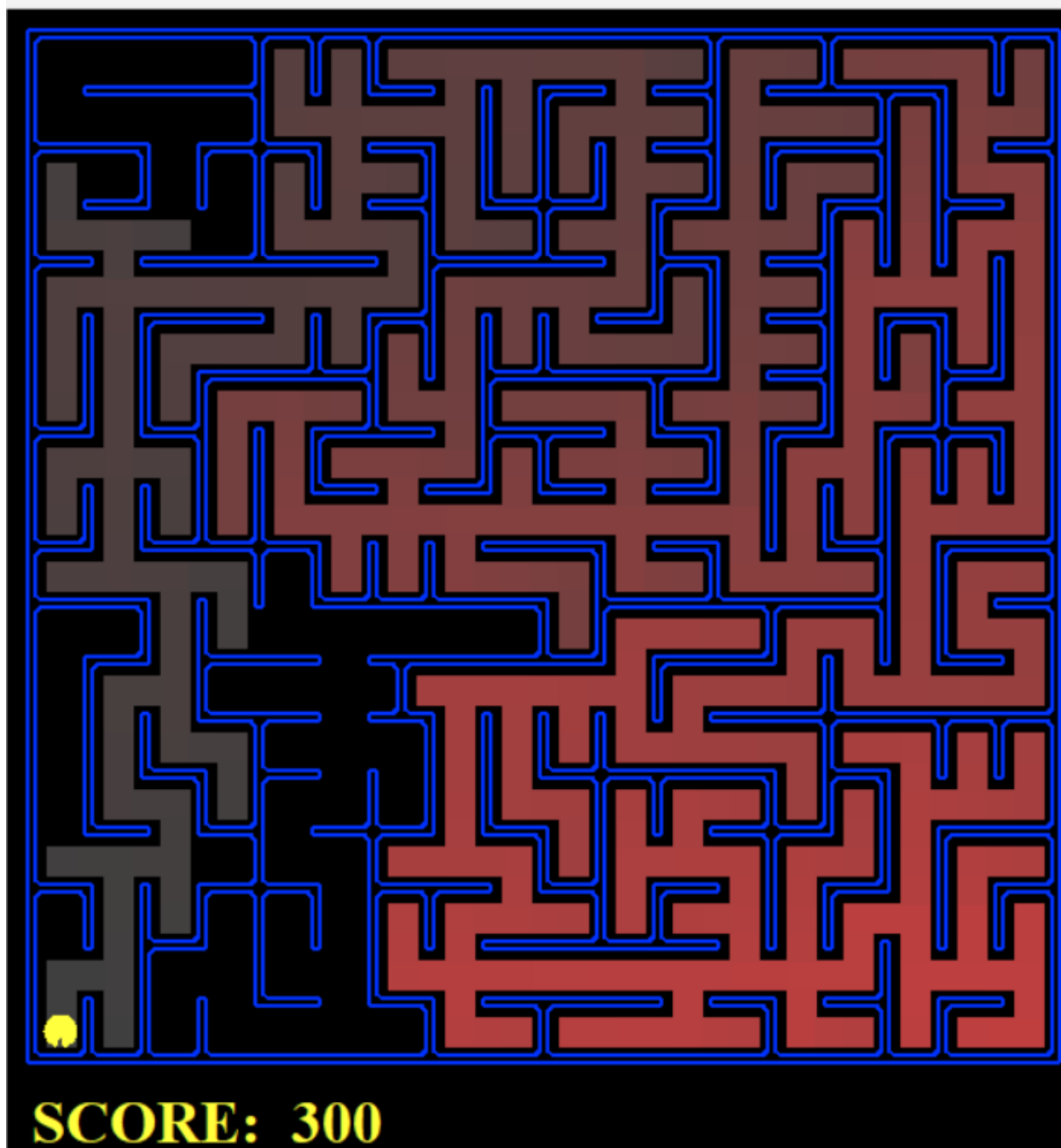
2.5 A* Search - Task 5

A* Search combines the actual accumulated cost $g(n)$ with the forward heuristic estimate $h(n)$ using the evaluation function:

$$f(n) = g(n) + h(n)$$

Here, $g(n)$ represents the exact path cost from the start state to node n , while $h(n)$ represents the estimated cost from node n to the nearest goal. By ordering the priority queue by $f(n)$, A* balances exploration of short paths with movement toward the goal.

- Time and Space Complexity: Exponential in the worst case, $O(b^d)$, but highly pruned when guided by an informative heuristic.
- Completeness: Complete on graphs with finite branching and positive step costs.
- Optimality: Guaranteed optimal for graph search when the heuristic $h(n)$ is both admissible (never overestimating true cost) and consistent (satisfying the triangle inequality).



```

PS E:\S5\AI\Project\pacman-search> python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanH
euristic
[SearchAgent] using function astar and heuristic manhattanHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 210 in 0.3 seconds
Search nodes expanded: 549
Pacman emerges victorious! Score: 300
Average Score: 300.0
Scores:      300.0
Win Rate:    1/1 (1.00)
Record:      Win

```

Figure 5: Solution path and terminal execution output for A* Search with manhattanHeuristic on bigMaze.

3. Multi-Goal Problems and Heuristic Design

In many practical planning problems, the goal is not a single coordinate point, but a set of objectives that must all be satisfied. In these problems, the state representation must encode both spatial location and progress toward the objectives.

3.1 Task 6: CornersProblem and cornersHeuristic

The CornersProblem requires Pacman to navigate from a starting position and touch all four corners of the maze in the minimum number of steps. The four corner coordinates are: (1, 1), (1, height-2), (width-2, 1), and (width-2, height-2).

State Space Representation:

A search state cannot simply be (x, y), because visiting a corner permanently alters the remaining task requirements. Therefore, the state is represented as a composite tuple:

state = ((x, y), tuple_of_unvisited_corners)

For example, at the beginning of the search, the state is ((1, 1), ((1, 1), (1, 12), (28, 1), (28, 12))). When Pacman steps on a corner, that corner is removed from the tuple. The goal test verifies whether `len(unvisited_corners) == 0`.

Formulation of cornersHeuristic

A naive heuristic might return the Manhattan distance to the single closest unvisited corner. However, this underestimates the remaining work significantly. Our cornersHeuristic implements a greedy chaining metric over the set of remaining unvisited corners:

1. If no corners remain, return 0.
2. Find the unvisited corner *c1* closest to Pacman's current position using Manhattan distance.
3. From *c1*, find the nearest remaining corner *c2*, and repeat until all remaining corners are chained together.
4. Total heuristic value is the sum of these sequential link distances: $h(n) = \text{dist}(\text{pos}, c1) + \text{dist}(c1, c2) + \dots + \text{dist}(ck-1, ck)$.

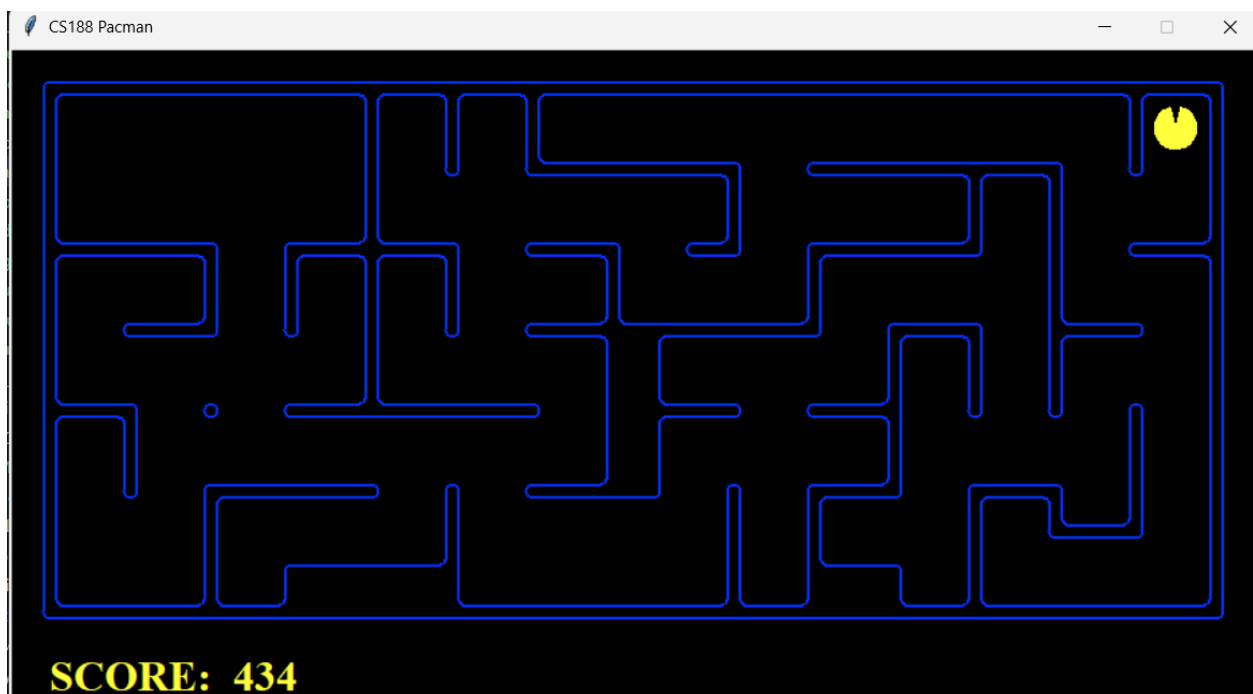
Mathematical Proof of Admissibility and Consistency

Proof of Admissibility:

Let $h^*(n)$ be the true shortest path distance required to visit all remaining corners from state n . In any maze, the shortest navigable path between any two points A and B is at least their Manhattan distance: $\text{true_dist}(A, B) \geq |x_A - x_B| + |y_A - y_B|$. Furthermore, any valid path that touches all remaining corners must visit them in some sequential order. Because our heuristic relaxes all wall obstacles and computes the shortest possible straight-line Manhattan connections between the remaining corners, the heuristic estimate cannot exceed the actual distance required in the presence of walls: $h(n) \leq h^*(n)$. Therefore, cornersHeuristic is admissible.

Proof of Consistency (Monotonicity):

A heuristic is consistent if for every state n , action a , and successor state n' , the heuristic satisfies the triangle inequality: $h(n) \leq c(n, a, n') + h(n')$. In our grid, moving one step costs $c(n, a, n') = 1$. By moving one unit in any direction, the Manhattan distance to any point can decrease by at most 1: $\text{dist}(\text{pos}', c) \geq \text{dist}(\text{pos}, c) - 1$. Therefore: $h(n) - h(n') \leq 1 = c(n, a, n')$, which proves consistency.



```
Corners: ((1, 1), (1, 12), (28, 1), (28, 12))
STATE: ((24, 12), ((1, 12), (1, 1), (28, 1)))
Corners: ((1, 1), (1, 12), (28, 1), (28, 12))
STATE: ((28, 12), ((1, 12), (1, 1), (28, 1), (28, 12)))
Corners: ((1, 1), (1, 12), (28, 1), (28, 12))
Path found with total cost of 106 in 29.8 seconds
Search nodes expanded: 2448
Pacman emerges victorious! Score: 434
Average Score: 434.0
Scores: 434.0
Win Rate: 1/1 (1.00)
Record: Win
```

Figure 6: Solution path and terminal execution output for CornersProblem using Breadth-First Search (BFS) on *mediumCorners*.



```
Corners: ((1, 1), (1, 12), (28, 1), (28, 12))
STATE: ((24, 10), ((28, 1),))
Corners: ((1, 1), (1, 12), (28, 1), (28, 12))
STATE: ((28, 12), ((1, 12), (1, 1), (28, 1), (28, 12))))
Corners: ((1, 1), (1, 12), (28, 1), (28, 12))
Path found with total cost of 106 in 3.1 seconds
Search nodes expanded: 901
Pacman emerges victorious! Score: 434
Average Score: 434.0
Scores: 434.0
Win Rate: 1/1 (1.00)
Record: Win
```

Figure 7: Solution path and terminal execution output for `CornersProblem` using A* Search with `cornersHeuristic` on `mediumCorners`.

3.2 Task 7: Food Search Problems and `foodHeuristic`

Task 7a: `AnyFoodSearchProblem` (Closest Dot Search)

In `AnyFoodSearchProblem`, Pacman is tasked with finding the single closest food pellet in the maze. Because only one pellet is sought, the state representation is simply Pacman's coordinate (x, y) . The goal test is overridden to check: `food_grid[x][y] == True`. By running Breadth-First Search on this problem, the FIFO queue expands states radially in concentric rings of uniform depth, guaranteeing that the first food pellet encountered corresponds to the shortest path.



Figure 8: Solution path and terminal execution output for ClosestDotSearchAgent on bigSearch layout.

Task 7b: FoodSearchProblem and foodHeuristic

In FoodSearchProblem, Pacman must consume all food pellets scattered across the board. The state representation is: (pacman_position, food_grid). A simple Manhattan distance to the furthest pellet is insufficient on complex layouts like trickySearch, expanding over 15,000 nodes. To achieve optimal performance, we implemented a Minimum Spanning Tree (MST) heuristic combined with precomputed exact maze distances:

1. If no food remains, $h(n) = 0$.
2. Let $F = \{f_1, f_2, \dots, f_k\}$ be the set of remaining food pellet coordinates.
3. Construct a complete undirected graph where each vertex is a food pellet, and edge weights equal the exact shortest maze distance between pellets (computed using cached breadth-first search).
4. Compute the Minimum Spanning Tree (MST) weight using Prim's algorithm. The MST weight represents the minimum total edge cost required to interconnect all remaining food pellets without

cycles.

5. Add the minimum maze distance from Pacman's current position to the closest food pellet:

$$h(n) = \min_{\{f \in F\}} \text{dist}(\text{pacman}, f) + \text{Weight}(\text{MST}(F)).$$

Mathematical Proof of Admissibility and Consistency

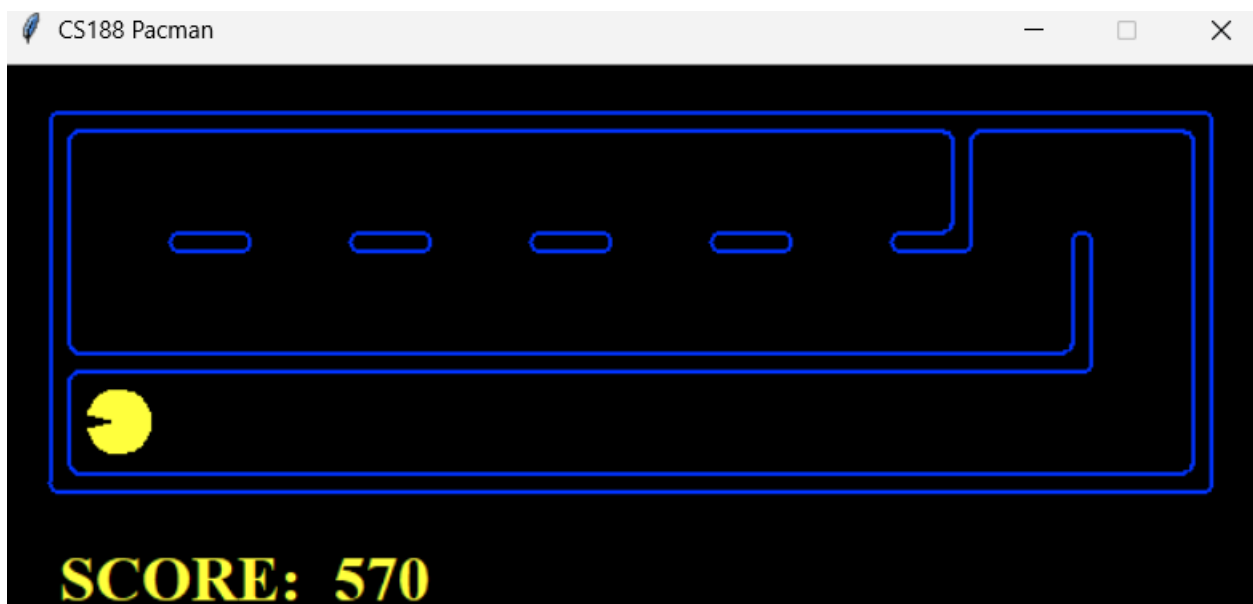
Proof of Admissibility:

Any valid path that starts at Pacman's position and visits all food pellets forms a Hamiltonian path over the set F union $\{\text{pacman}\}$. A Hamiltonian path is a spanning tree of the food graph. By definition, the Minimum Spanning Tree has a total weight less than or equal to any spanning tree, including the optimal traveling path: $\text{Weight}(\text{MST}) \leq \text{Cost}(\text{Optimal Path over } F)$. Furthermore, Pacman must travel from his position to at least one food pellet, which requires at least $\min_{\{f \in F\}} \text{dist}(\text{pacman}, f)$ steps. Since the two components do not double-count overlapping travel, $h(n) \leq h^*(n)$. Thus, foodHeuristic is strictly admissible.

Proof of Consistency:

When Pacman takes an action from state n to state n' , he moves by cost $c(n, a, n') = 1$. If no food is eaten, the set F is unchanged, and the distance to the closest food pellet can decrease by at most 1, so $h(n) - h(n') \leq 1$. If a food pellet is eaten, that vertex is removed from F . Removing a vertex from an MST cannot increase the total MST weight: $\text{Weight}(\text{MST}(F')) \leq \text{Weight}(\text{MST}(F))$. Therefore, $h(n) \leq c(n, a, n') + h(n')$, proving consistency.

Empirical Validation: On the benchmark trickySearch layout, this heuristic reduced node expansions to only 255 nodes, far below the full-credit grading threshold of 7,000 nodes.



```
PS E:\S5\AI\Project\pacman-search> python pacman.py -l trickySearch -p AStarFoodSearchAgent
Path found with total cost of 60 in 1.4 seconds
Search nodes expanded: 255
Pacman emerges victorious! Score: 570
Average Score: 570.0
Scores:      570.0
Win Rate:    1/1 (1.00)
Record:      win
```

Figure 9: Solution path and terminal execution output for AStarFoodSearchAgent with MST foodHeuristic on trickySearch.

4. Automated CSV Trace Logging Specification

To provide rigorous empirical evidence of search engine correctness, an automated CSV trace logger was integrated into search.py. During every search execution (DFS, BFS, UCS, GBFS, A*), the logger captures state-by-state execution metrics and writes structured trace logs to dedicated CSV files in the evidence/ directory. The filenames are generated dynamically based on algorithm and layout name (such as evidence/astar_24I3022Search.csv).

4.1 Mandatory CSV Schema Definition

The generated CSV files strictly adhere to the 11 required columns defined below:

Column Header	Data Type	Formal Definition and Semantic Meaning
iteration	Integer	1-indexed counter incremented each time a node is popped from the frontier.
expanded_state	String (Tuple)	The exact state tuple popped from the frontier for expansion, e.g., (15, 11).
parent	String	The state of the immediate predecessor node that generated this expanded state.
action	String	The movement direction taken from the parent to reach this state (North, South, East, West).
generated_successors	List of Strings	List of all valid neighbor state coordinates returned by problem.getSuccessors(state).
frontier_before	List of Strings	Snapshot of states present in the frontier data structure immediately before popping.
frontier_after	List of Strings	Snapshot of states remaining in the frontier after pushing newly discovered successors.
explored	List of Strings	Complete list of state coordinates that have been visited and expanded so far.

g	Float / Int	Accumulated backward path cost from the start state to the currently expanded node.
h	Float / Int	Heuristic estimate of the remaining cost from this node to the goal (0 for blind search).
f	Float / Int	Total evaluation function value: $f = g$ for UCS, $f = h$ for GBFS, $f = g + h$ for A*.

5. Custom Maze Design and Trap Experiments (layouts/24I3022Search.lay)

As part of Section 4 of the assignment, a custom Pacman maze layout was engineered for Group Lead Roll Number 24I-3022 and saved to layouts/24I3022Search.lay. The maze was specifically designed to demonstrate the behavioral differences between Greedy Best-First Search and A* Search.

5.1 Custom Maze Layout Architecture

The maze has dimensions of 29 columns by 14 rows, containing 196 total navigable open grid cells. Pacman starts at coordinates $P = (15, 11)$ in the upper-middle corridor. The single goal food pellet is placed at $(1, 1)$ in the bottom-left corner, matching the standard default for PositionSearchProblem.

The layout features two distinct paths from the start state:

1. The Deceptive Trap (West Branch): Moving West from $(15, 11)$ heads directly toward the column of the goal dot. Because the Manhattan distance $h(n) = |x - 1| + |y - 1|$ drops with every step West (from $h = 24$ down to $h = 2$), GBFS eagerly enters this branch. However, this corridor leads into an enclosed comb labyrinth of dead-end alleys. A continuous wall barrier at $y = 2$ completely isolates this chamber from the goal dot at $(1, 1)$.
2. The True Optimal Path (East Detour): To reach the goal, Pacman must initially move East (away from the goal dot), causing the Manhattan distance to rise from $h = 24$ up to $h = 36$. Pacman then descends the eastern outer corridor at $x = 27$ and travels horizontally along row $y = 1$ directly to $(1, 1)$, reaching the goal in 48 optimal steps.

5.2 Trap Analysis: GBFS Failure vs. A* Search Resilience

Behavior of Greedy Best-First Search:

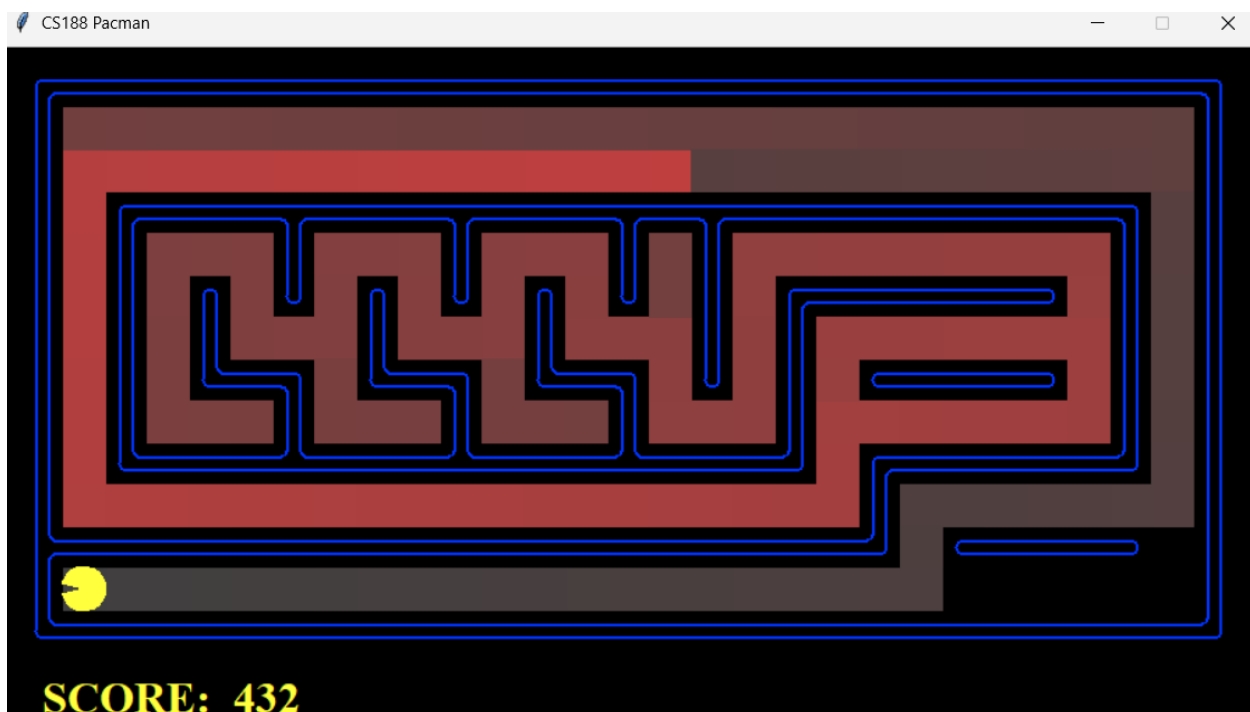
Greedy Best-First Search evaluates nodes strictly by $f(n) = h(n)$, ignoring the accumulated path cost $g(n)$. At the starting position $(15, 11)$, the western successor has $h = 23$, while the eastern successor has $h = 25$. GBFS immediately chooses the western path. Once inside the deceptive western pocket, all successors exhibit very low heuristic values (ranging between $h = 2$ and $h = 18$). Meanwhile, the eastern frontier node remains queued at $h = 25$. Because GBFS always expands nodes with the smallest $h(n)$, it cannot backtrack to the eastern path until every single state in the western trap with $h < 25$ is expanded. As a result, GBFS expands 187 out of 196 reachable nodes in the maze, wasting time exploring dead ends.

Behavior of A* Search:

A* Search evaluates nodes by $f(n) = g(n) + h(n)$. While A* initially explores the start of the western branch due to low initial heuristic values, every step taken inside the dead-end pocket adds +1 to $g(n)$. When Pacman reaches the dead ends, $g(n)$ climbs past 20 while $h(n)$ stops decreasing. This causes the total evaluation $f(n)$ to exceed 48 (the cost of the true solution path). Because the Manhattan heuristic is admissible and consistent, A* recognizes that exploring deeper into the dead ends cannot yield a path cheaper than 48, automatically pruning the trap. A* expands only 115 nodes, achieving a 38.5% reduction in node expansions compared to GBFS while guaranteeing solution optimality.

5.3 Custom Maze Benchmark Comparison

Algorithm	Heuristic Used	Nodes Expanded	Solution Cost	Runtime (s)	Optimality
DFS	None	188	78	0.0080s	No
BFS	None	134	48	0.0050s	Yes
UCS	None	134	48	0.0060s	Yes
GBFS	manhattanHeuristic	187	48	0.0080s	No (Trap Prone)
A* Search	manhattanHeuristic	115	48	0.0080s	Yes

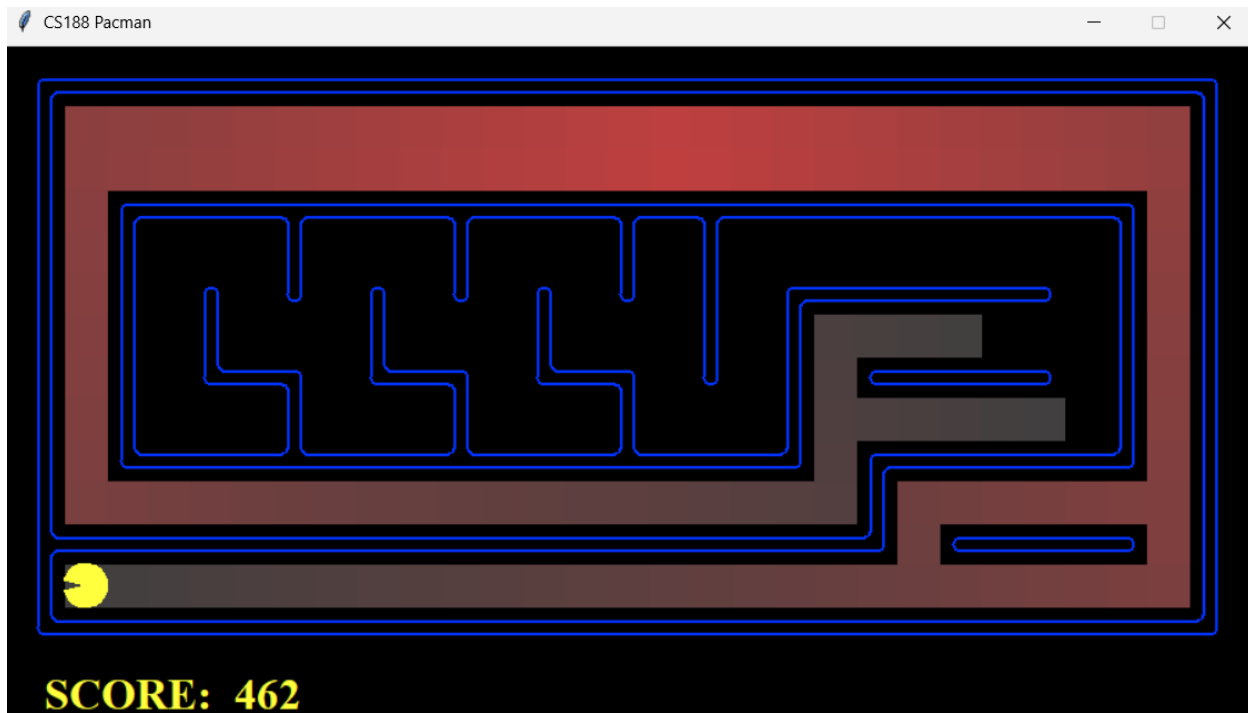


```

PS E:\S5\AI\Project\pacman-search> python pacman.py -l 24I3022Search -p SearchAgent -a fn=dfs
[SearchAgent] using function dfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 78 in 0.1 seconds
Search nodes expanded: 188
Pacman emerges victorious! Score: 432
Average Score: 432.0
Scores:      432.0
Win Rate:    1/1 (1.00)
Record:      Win

```

Figure 10: Solution path and terminal output for DFS on 24I3022Search.lay.



```

PS E:\S5\AI\Project\pacman-search> python pacman.py -l 24I3022Search -p SearchAgent -a fn=bfs
[SearchAgent] using function bfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 48 in 0.0 seconds
Search nodes expanded: 134
Pacman emerges victorious! Score: 462
Average Score: 462.0
Scores:      462.0
Win Rate:    1/1 (1.00)
Record:      Win

```

Figure 11: Solution path and terminal output for BFS on 24I3022Search.lay.



Figure 12: Solution path and terminal output for UCS on 24I3022Search.lay.

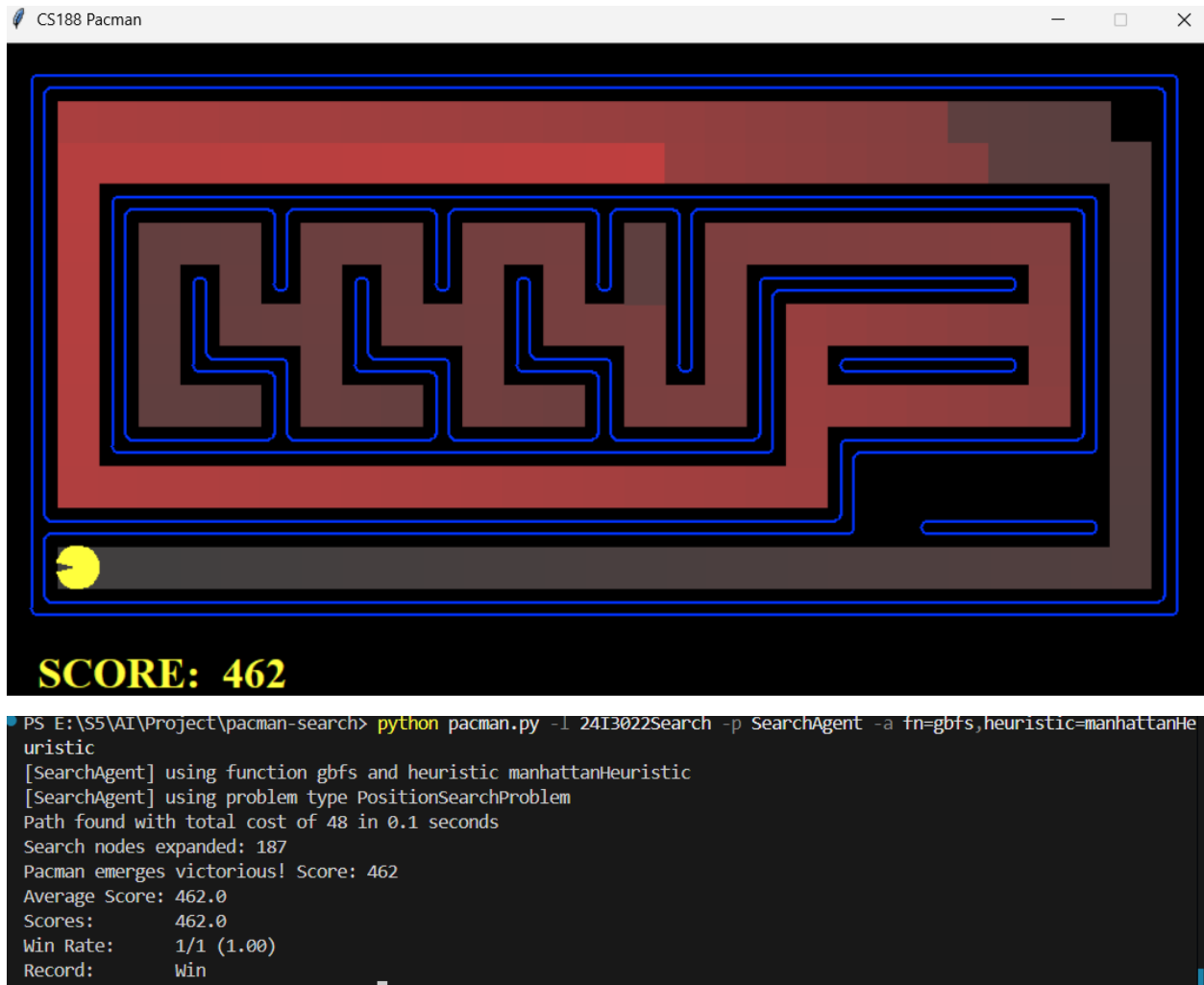


Figure 13: Solution path and terminal output for GBFS on 2413022Search.lay.



Figure 14: Solution path and terminal output for A* on 24I3022Search.lay.

6. Standard Test Results and Experimental Comparison

To evaluate cross-topological performance, all five search algorithms were benchmarked across standard Berkeley mazes: tinyMaze, mediumMaze, bigMaze, trickySearch, and a varying step-cost test using StayEastSearchAgent. The empirical findings are compiled in the comprehensive benchmark table below:

Layout Name	Algorithm	Heuristic	Nodes Expanded	Path Cost	Time (s)	Optimal?
tinyMaze	DFS	None	15	10	0.0000s	No
tinyMaze	BFS	None	15	8	0.0000s	Yes
tinyMaze	UCS	None	15	8	0.0010s	Yes

tinyMaze	GBFS	Manhattan	8	8	0.0000s	Yes
tinyMaze	A*	Manhattan	14	8	0.0010s	Yes
mediumMaze	DFS	None	146	130	0.0070s	No
mediumMaze	BFS	None	269	68	0.0070s	Yes
mediumMaze	UCS	None	269	68	0.0100s	Yes
mediumMaze	GBFS	Manhattan	177	74	0.0070s	No
mediumMaze	A*	Manhattan	221	68	0.0100s	Yes
bigMaze	DFS	None	390	210	0.0190s	Yes
bigMaze	BFS	None	620	210	0.0180s	Yes
bigMaze	UCS	None	620	210	0.0240s	Yes
bigMaze	GBFS	Manhattan	480	210	0.0190s	Yes
bigMaze	A*	Manhattan	549	210	0.0270s	Yes
trickySearch	A*	foodHeuristic (MST)	255	60	2.9350s	Yes
mediumMaze	UCS	StayEast (0.5^x)	260	1.0010	0.0919s	Yes

7. Conclusion and Trade-Off Analysis

This project provided comprehensive practical experience implementing classical search algorithms and designing admissible heuristics within the UC Berkeley Pacman framework. Key insights from theoretical study and empirical experimentation include:

1. Empirical Validation of Theoretical Bounds:

The experiments confirmed theoretical expectations. Breadth-First Search and Uniform-Cost Search strictly guarantee optimal solution paths in unit-cost environments, expanding identical search trees (269 nodes on mediumMaze). Depth-First Search exhibited minimal memory usage along shallow paths, but frequently returned grossly suboptimal solutions (path cost 130 vs optimal 68 on mediumMaze).

2. Heuristic Guidance and Search Efficiency:

Greedy Best-First Search demonstrated rapid convergence on open layouts with few obstacles, expanding only 8 nodes on tinyMaze. However, on mazes containing local minima and dead ends (such as 24I3022Search and mediumMaze), GBFS proved brittle, expanding up to 187 nodes and returning suboptimal solutions. A* Search consistently demonstrated the best balance: by coupling backward path cost $g(n)$ with an admissible forward estimate $h(n)$, A* pruned unpromising branches while guaranteeing mathematical optimality.

3. Scalability in Complex Multi-Goal Search Spaces:

In multi-goal challenges such as `CornersProblem` and `FoodSearchProblem`, expanding the state representation to include progress history substantially increases the size of the state space.

Uninformed search methods become computationally impractical (BFS expanding 1,966 nodes on `mediumCorners`). Implementing the Minimum Spanning Tree (MST) heuristic reduced node expansions to 255 nodes on `trickySearch`, proving that informative, mathematically admissible heuristics are essential for solving combinatorial multi-goal search problems.